AI in Robocode: Printable Technical Notes

Source deck: Tex/robocode_ai_presentation.tex

1) Objectives

Slide: Tex/robocode_ai_presentation.tex

1. Define the state representation used by the bot.
2. Define the action space.
3. Explain DQN as a value-based RL method.
4. Show target, loss, and update mechanics.
5. Measure performance with train vs eval runs.

2) Architecture Overview

Slide: Tex/robocode_ai_presentation.tex

Pipeline in one process: 1. Robocode events feed DQNBot. 2. DQNBot encodes state. 3. DQNAgent selects action using epsilon-greedy policy. 4. Bot executes action and accumulates reward from events. 5. Transition is stored in replay memory. 6. Agent samples minibatches and updates policy and target networks.

Primary code locations: 1. Bot loop and transition staging: Jacob/Jacob3_0/dqn_bot.py 2. Agent action selection: Jacob/Jacob3_0/dqn_agent.py 3. Replay memory implementation: Jacob/Jacob3_0/dqn_agent.py

3) State Space (16 Features)

Slide: Tex/robocode_ai_presentation.tex Implementation: Jacob/Jacob3_0/dqn_bot.py

Equation:

$$s_t \in \mathbb{R}^{16}$$

Feature groups used in code: 1. Self status: normalized energy, speed, heading sin/cos. 2. Position context: normalized x/y and wall-relative distances. 3. Enemy context: bearing sin/cos, normalized distance, enemy energy. 4. Temporal features: delta bearing and delta distance. 5. Weapon readiness: normalized gun heat. 6. Freshness bit: recent scan indicator.

Angle wrap handling: 1. Bearing normalization to [-pi, pi]: Jacob/Jacob3_0/dqn_bot.py 2. Delta bearing normalization: Jacob/Jacob3_0/dqn_bot.py

4) Action Space (7 Discrete Actions)

Slide: Tex/robocode_ai_presentation.tex Constants: Jacob/Jacob3_0/dqn_bot.py Execution map: Jacob/Jacob3_0/dqn_bot.py

Equation:

$$a_t \in \{0, 1, 2, 3, 4, 5, 6\}$$

Action mapping: 1. 0 strafe left 2. 1 strafe right 3. 2 forward 4. 3 backward 5. 4 fire power 1.0 6. 5 fire power 2.0 7. 6 fire power 3.0

5) Reward Design and Event Wiring

Slide summary: Tex/robocode_ai_presentation.tex

Reward-related code: 1. Bullet damage formula helper: Jacob/Jacob3_0/dqn_bot.py 2. Hit by bullet penalty: Jacob/Jacob3_0/dqn_bot.py 3. Bullet hit reward: Jacob/Jacob3_0/dqn_bot.py 4. Wall hit penalty: Jacob/Jacob3_0/dqn_bot.py 5. Fire-cost shaping term: Jacob/Jacob3_0/dqn_bot.py 6. Terminal win/loss reward: Jacob/Jacob3_0/dqn_bot.py

Interpretation: 1. Dense shaping gives short-horizon guidance. 2. Terminal +/-1 anchors long-horizon objective. 3. Fire-cost discourages low-value shot spam.

6) DQN Policy and Exploration

Slide: Tex/robocode_ai_presentation.tex

Greedy policy equation:

$$\pi(s_t) = \arg\max_a Q_\theta(s_t, a)$$

Exploration schedule equation:

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \exp\left(-\frac{\text{steps}}{\text{decay}}\right)$$

Code references: 1. Epsilon-greedy branch: Jacob/Jacob3_0/dqn_agent.py 2. Epsilon decay computation: Jacob/Jacob3_0/dqn_agent.py 3. Eval mode fixed epsilon: Jacob/Jacob3_0/dqn_agent.py 4. MLP network 16->128->128->7: Jacob/Jacob3_0/dqn_agent.py

7) Target, Loss, and Soft Update

Slide: Tex/robocode_ai_presentation.tex

Bootstrapped target:

$$y_t = r_t + \gamma(1 - d_t) \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a')$$

Loss:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\text{SmoothL1}\left(Q_\theta(s_t, a_t), y_t\right)\right]$$

Soft target update:

$$\bar{\theta} \leftarrow \tau\,\theta + (1 - \tau)\,\bar{\theta}$$

Code references: 1. Train step body: Jacob/Jacob3_0/dqn_agent.py 2. Terminal masking and target construction: Jacob/Jacob3_0/dqn_agent.py 3. SmoothL1 loss call: Jacob/Jacob3_0/dqn_agent.py 4. Gradient clipping: Jacob/Jacob3_0/dqn_agent.py 5. Soft update implementation: Jacob/Jacob3_0/dqn_agent.py

8) Episode Loop Snapshot (Train vs Eval)

Slide snippet: Tex/robocode_ai_presentation.tex Loop implementation: Jacob/Jacob3_0/dqn_bot.py

Per-step sequence: 1. Encode current state. 2. Select action. 3. Push previous transition with accumulated step reward. 4. Execute action. 5. Save current state/action for next transition. 6. Rotate radar for refreshed scan events.

Mode behavior: 1. Train mode enables replay writes and optimization. 2. Eval mode disables online learning and can fix epsilon at 0.0.

Switch points: 1. Bot mode selection: Jacob/Jacob3_0/dqn_bot.py 2. Agent mode methods: Jacob/Jacob3_0/dqn_agent.py

9) Metrics, Logging, and Reported Results

Results slide: Tex/robocode_ai_presentation.tex

Win rate equation shown in deck:

$$\text{WinRate} = \frac{6264}{11420} = 0.549$$

Bot log output structure: 1. Episode result dict fields: Jacob/Jacob3_0/dqn_bot.py 2. JSONL append function: Jacob/Jacob3_0/dqn_bot.py

Key fields: 1. episode 2. won 3. total_reward 4. steps 5. mode 6. epsilon 7. buffer_size 8. win_rate 9. training_steps

10) Interpretation Notes for Discussion

Slides: Tex/robocode_ai_presentation.tex

Current behavior summary: 1. Strong performance against simpler or stationary behavior patterns. 2. Weakness against movement-heavy pressure bots, especially Walls and SpinBot. 3. Train-eval discrepancy suggests curriculum imbalance and overfitting risk.

Practical next changes from deck: 1. Increase curriculum sampling weight for weak-match opponents. 2. Strengthen wall-pressure penalties. 3. Add wall-escape action patterns. 4. Keep eval isolated with fixed epsilon and no online updates.

11) Quick Print Checklist

1. Verify formulas above match slide equations.
2. Verify code links still point to current line numbers after edits.
3. Print this file directly from VS Code or export to PDF.

12) What Is Not On The Slides (But Matters In Practice)

12.1 Reward scaling asymmetry

The reward terms are not symmetric, which can bias policy behavior: 1. Bullet hit reward is +0.02 times damage. 2. Hit-by-bullet penalty is -0.025 times damage. 3. Wall collision is a flat -0.05 per event. 4. Firing has a small cost of -0.01 when a shot is actually fired.

Code locations: 1. Jacob/Jacob3_0/dqn_bot.py 2. Jacob/Jacob3_0/dqn_bot.py 3. Jacob/Jacob3_0/dqn_bot.py 4. Jacob/Jacob3_0/dqn_bot.py

Implication: 1. Getting hit hurts more than hitting helps, so safer positioning may dominate aggression unless terminal rewards offset this.

12.2 Freshness feature default behavior

When no recent scan exists, enemy features are filled with defaults instead of zeros: 1. distance defaults to half arena diagonal. 2. enemy energy defaults to 100. 3. freshness bit is 0.

Code location: Jacob/Jacob3_0/dqn_bot.py

Implication: 1. The model learns an implicit prior for unknown enemy state rather than a true missing-value encoding.

12.3 Training starts only after replay warmup

Optimization does not begin immediately: 1. transitions are appended every step. 2. training starts only when replay size reaches batch size.

Code location: Jacob/Jacob3_0/dqn_agent.py

Implication: 1. Early episodes mainly populate replay memory, so early metrics can look noisy and undertrained.

12.4 Terminal transition handling

Terminal transitions write a zero next-state vector and done flag true.

Code location: Jacob/Jacob3_0/dqn_bot.py

And in training, done transitions are masked to remove bootstrap value:

Code location: Jacob/Jacob3_0/dqn_agent.py

Implication: 1. This matches standard episodic DQN and avoids leaking value from synthetic terminal next states.

12.5 Eval mode safety: where learning is actually blocked

The bot still calls push transition in the loop, but the agent no-ops when training is disabled.

Code locations: 1. Bot push call: Jacob/Jacob3_0/dqn_bot.py 2. Agent training guard: Jacob/Jacob3_0/dqn_agent.py

Implication: 1. Evaluation remains read-only for learning even if loop logic is shared.

12.6 Important naming mismatch

The method docstring says Double DQN, but the target is computed as max over target-net values directly.

Code location: Jacob/Jacob3_0/dqn_agent.py

Current target calculation: 1. select and evaluate next action from target net only. 2. this is classic DQN target-net bootstrapping, not Double DQN action-selection split.

Implication: 1. Slightly higher overestimation risk than true Double DQN.

13) Source Snippets For Fast Review

13.1 Main step loop snippet

From Jacob/Jacob3_0/dqn_bot.py

```
while self.running:
    self.local_tick += 1

    state = self._encode_state()
```

```
    action = self.agent.select_action(state)

    if self.prev_state is not None and self.prev_action is not None:
        self.agent.push_transition(
            self.prev_state, self.prev_action, state, self.step_reward, done=False
        )
        self.episode_reward += self.step_reward

    self.step_reward = 0.0
    self._execute_action(action)

    self.prev_state = state.copy()
    self.prev_action = action
```

13.2 Reward event snippet

From Jacob/Jacob3_0/dqn_bot.py

```
def on_hit_by_bullet(self, hit_by_bullet_event: HitByBulletEvent) -> None:
    power = float(getattr(getattr(hit_by_bullet_event, "bullet", None), "power", 1.0))
    damage = _bullet_damage(power)
    self.step_reward -= 0.025 * damage

def on_bullet_hit(self, bullet_hit_bot_event) -> None:
    power = float(getattr(getattr(bullet_hit_bot_event, "bullet", None), "power", 1.0))
    damage = _bullet_damage(power)
    self.step_reward += 0.02 * damage
```

13.3 Training target and loss snippet

From Jacob/Jacob3_0/dqn_agent.py

```
with torch.no_grad():
    next_q_values = self.target_net(next_states).max(dim=1)[0]
    next_q_values[dones] = 0.0
    targets = rewards + self.gamma * next_q_values

loss = nn.SmoothL1Loss()(q_values, targets)
```

14) Suggested Appendix For Q and A

If asked why performance drops against specific bots, anchor discussion to these points: 1. Opponent distribution mismatch between training and evaluation pools. 2. Reward scaling may under-incentivize risky but necessary offensive actions. 3. Action space has no explicit wall-escape macro action. 4. Current target calculation is standard DQN style and may overestimate values in some states.